
Contracted Hierarchies for Long-Horizon Meta-Agent Management

Anonymous Author(s)

Affiliation

Address

email

1 Submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026), workshop: Managing Agents
2 that Manage Agents. Do not distribute.

Abstract

3
4 Researchers and developers increasingly compose persistent chat, research, and coding-agent
5 surfaces into informal long-horizon workflows, while retaining most coordination in human
6 attention. We present a package-resident meta-agent management protocol that formalizes this
7 pattern. Main is a persistent planning and decision chat; Theory and Experiment are optional
8 specialist reasoning workstreams; Main authors one durable multi-view intent contract (DIC) and
9 the initial execution prompt stack (EPS) for each approved Phase; a repository-aware coding-agent
10 Orchestrator validates and model-tunes that prompt graph for the active surface, executes bounded
11 Workers/subagents, and may issue contract-preserving repair EPS revisions. The DIC also acts as
12 a human-intelligibility interface that preserves one Primary Aim and prioritized requests while
13 hiding routine execution noise. Verification returns accept, repair, replan, or material escalation.
14 Under complete read/write declarations and versioned evidence, a repair need invalidate only its
15 dynamic dependency cone, and that cone is worst-case tight; we also bound fault survival under
16 imperfect verification and characterize a checkpoint-cost trade-off. We introduce human-attention
17 intensity—supervisory attention per unit of verified progress—as an evaluation target rather than
18 an established performance result. The reference implementation provides reconstructable lineage
19 and model-tuned execution controls, while a frozen matched-budget study is designed to test
20 whether the hierarchy improves trustworthy long-horizon progress without requiring proportional
21 growth in human supervision.

1 Introduction

22
23 Manager–worker agents, explicit plans, reflection, memory, and workflow optimization are established
24 ideas [Yao et al., 2023b, Wu et al., 2023, Hong et al., 2024, Hu et al., 2025]. In practice, many
25 researchers already build an informal meta-workflow: a persistent chat plans, a research surface
26 gathers evidence, and a coding agent implements and runs experiments. The unresolved systems
27 problem is that state, routing, and quality control still live largely in the researcher’s attention. As
28 execution extends across fresh contexts, files, and subagents, the human repeatedly reconstructs what
29 is current, which branch depends on what, which result is verified, and when a local repair changes a
30 downstream claim.

31 We study an operational question for meta-agent systems: *what persistent object lets a manager refine*
32 *intent, supervise bounded workers, and recover locally while keeping human attention concentrated*
33 *on material decisions rather than routine coordination?* Our answer is a contracted hierarchy stored
34 in the working package:

$$\text{Goal} \succeq \text{Plan} \succeq \text{Phase} \succeq \text{DIC} \succeq \text{EPS} \succeq \text{Prompt/Run} \succeq \text{Action}.$$

35 The durable levels encode semantic authority; disposable EPS attempts encode current operations.
36 The DIC is also a human-comprehension boundary: one Primary Aim, prioritized requests, the
37 durable dependency/parallelism graph, evidence requirements, and material stops remain readable
38 even when the coding-agent execution tree becomes large.

39 This structure targets two semantic drifts common in long-horizon AI-assisted research. *Constraint–*
40 *objective inversion* occurs when a safeguard such as “avoid unsupported claims” gradually becomes

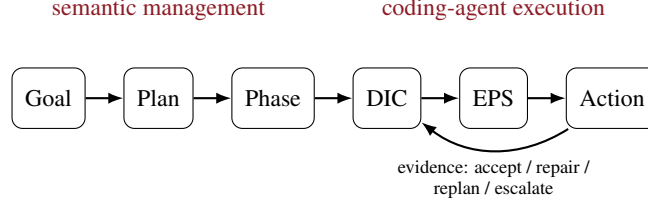


Figure 1: Durable semantic refinement and disposable execution. Replan travels upward; local repair regenerates EPS under the same DIC.

the effective optimization objective, producing defensive but scientifically weaker work. *Saliency flattening* occurs when many locally reasonable branches receive similar weight until the programme loses a clear central contribution. The contracted hierarchy distinguishes the Primary Aim from constraints, supporting questions, and optional branches so a meta-agent can adapt the route without silently changing the destination.

Our contribution is not hierarchical planning alone. It is the integrated protocol of (i) persistent intent and authority, (ii) explicit semantic refinement and saliency, (iii) bounded coding-agent execution with model-tuned prompts and subagents, (iv) environment-grounded evidence gates, (v) dependency-local repair or upward replanning, and (vi) an explicit human-attention objective. We contribute:

1. an executable state machine and reference implementation separating Main-level semantic management from coding-agent Orchestration;
2. formal results stating when contract composition and dependency-local recovery are sound, how imperfect verification affects detection, and why incomplete evidence precludes universal correctness;
3. a human-attention intensity measure and frozen experimental design that isolate hierarchy/repair from extra inference while measuring both task quality and supervisory burden.

No large confirmatory agent study is claimed. The paper treats long-horizon and attention benefits as hypotheses, reports implementation validation only, and defines the evidence required to upgrade the claims.

2 Management protocol

2.1 Human and machine timescales

A **Plan** is a human-facing DAG of several phases, with an approved goal, owners, dependencies, acceptance evidence, and checkpoint interval. A **Phase** is a coherent research milestone, not ordinarily an approval boundary. A **DIC** is one multi-view Phase contract: README exposes the Primary Aim, priority structure, authority, and compact orientation; REQUESTS records prioritized outcomes and acceptance targets; PLAN records the durable prompt/dependency/parallel-subagent execution graph. An **EPS** is a disposable just-in-time coding-agent instantiation of ready DIC plan nodes for the current state and attempt. An **Action** is a tool call, edit, search, test, or experiment linked to its prompt/run and evidence record.

Main is the persistent user-facing semantic manager: it owns Plan authority, one whole-Phase DIC, specialist reconciliation, material decisions, and publication/closure. Theory and Experiment are optional specialist reasoning workstreams; Research is advisory evidence returning to Main. Main also authors the initial EPS/prompt graph for the Phase. Operational execution is delegated to a distinct repository-aware coding-agent **Orchestrator**, which validates that EPS, compiles its prompt wording for the actual model/surface, routes bounded Workers/subagents, manages safe parallelism and write ownership, integrates returns, performs contract-preserving repair through superseding EPS revisions, and dispatches verification. Routine execution should remain below the human-attention boundary unless scientific meaning or decisive evidence changes.

2.2 Human-attention objective

Let A_H be recorded human supervisory attention (measured duration when available, otherwise intervention counts by class) and let $P_V = \sum_v q_v z_v$ be prespecified weighted progress over evidence-

82 accepted milestones. We define human-attention intensity

$$\rho_H = \frac{A_H}{P_V + \varepsilon}.$$

83 The workflow targets lower ρ_H by moving routine routing, state reconstruction, local repair, and
 84 evidence bookkeeping into durable machine-managed surfaces. This is a benchmark-relative objective,
 85 not a universal productivity score or an established empirical gain. Human judgement remains
 86 mandatory for material scientific, authority, release, and external-action decisions.

87 2.3 State and recovery

88 A small canonical index, `.ai-workflow/STATE.json`, stores the active Plan, phase summaries,
 89 checkpoints, escalations, roles, and evidence pointers. Multi-view DIC packages, EPS prompt/run
 90 instantiations, Action records, and mandatory flat Phase reports are stored beneath the Plan/Phase
 91 path and linked by identifiers and hashes. A fresh context can reconstruct the approved objective,
 92 current phase, lineage, and unresolved governance events from the package alone. Narrative handoffs
 93 are views rather than competing authorities.

94 2.4 Verification loop

95 Verification consumes the DIC, state, and evidence and returns `{accept, repair, replan, escalate}`.
 96 Repair preserves the DIC and regenerates EPS. Replan changes the decomposition, acceptance
 97 meaning, or protocol. Escalation is immediate for material objective, authority, scientific-claim,
 98 protocol, destructive/external, release/submission, paid-compute, or credential changes. Routine local
 99 failures remain machine decisions.

100 3 Formal analysis

101 3.1 Refinement and composition

102 Associate every hierarchy object X with admitted traces $\llbracket X \rrbracket$. For an abstraction map ρ , define
 103 $X \preceq Y$ when $\rho(\llbracket X \rrbracket) \subseteq \llbracket Y \rrbracket$. Refinement chains preserve the parent intent only when the maps
 104 compose and each inclusion is sound; hierarchy does not establish those facts automatically.

105 For a Plan DAG, let each phase implement a sound pre/post contract, let predecessor postconditions
 106 imply successor preconditions, require noninterference for unordered writes, and let terminal postcon-
 107 ditions imply the global goal. Standard induction over a topological order then yields compositional
 108 satisfaction. This identifies the real assumptions hidden by informal “manager decomposes task”
 109 descriptions.

110 3.2 Dependency-local recovery

111 Let phase v declare read keys R_v and write keys W_v ; dependency edges include every possible data
 112 flow. If a repair changes keys Δ , define

$$B(\Delta) = \{v : R_v \cap \Delta \neq \emptyset\}, \quad \mathcal{I}(\Delta) = \text{Reach}(B(\Delta)).$$

113 **Theorem 1** (Local recovery). *If phase outputs and evidence depend only on declared inputs,*
 114 *tool/version parameters, predecessor outputs, and recorded randomness; dependencies are com-*
 115 *plete; and evidence is version-bound, then retaining completed outputs outside $\mathcal{I}(\Delta)$ preserves their*
 116 *contract validity.*

117 *Proof sketch.* Outside the cone, a node neither reads a changed key nor depends on a changed
 118 predecessor. Topological induction preserves every declared input and therefore its output/evidence
 119 validity. Full proof is in the technical report. $\square$

120 The cone is worst-case tight: choose a direct reader to copy a changed bit and each descendant to
 121 propagate it. Every reachable node can then become invalid. This is an agent-workflow adaptation of
 122 dependency-graph and incremental-build locality, not a claim that those underlying ideas are new
 123 [Mokhov et al., 2018].

Property	Validation artifact
One Plan authorizes several phases	unit test
Ordinary phase progression without approval ceremony	unit test
Periodic checkpoint trigger	unit test
Immediate material escalation	unit test
Action→EPS→DIC→Phase→Plan lineage	unit test + recovery audit
Main/Theory/Experiment ownership and aliases	unit test
Fresh-session package recovery	unit test + CLI audit
Reasonable v5.6.31 compatibility	migration-index test

Table 1: Engineering conformance coverage. The release checklist resolves actual build logs; these are not comparative agent results.

3.3 Imperfect verification and checkpoints

If the conditional probability of missing a persisting observable fault at each gate is at most $\alpha < 1$, survival through k gates is at most α^k , and expected detection delay is at most $1/(1 - \alpha)$ gate intervals. In a stationary model with checkpoint cost c_v , false-rejection probability β and cost c_f , fault rate λ , and per-phase repair cost c_r , expected overhead at interval h is

$$U(h) = \frac{n(c_v + \beta c_f)}{h} + \lambda n c_r h \frac{1 + \alpha}{2(1 - \alpha)},$$

with continuous minimizer $h^* = \sqrt{2(c_v + \beta c_f)(1 - \alpha)/(\lambda c_r(1 + \alpha))}$. This motivates configurable checkpoints rather than per-phase human approval or a universal fixed cadence [Young, 1974, Daly, 2006].

3.4 Observability limit

If evidence map O assigns the same observation to states s_+ and s_- while the goal is true only at s_+ , every evidence-only verifier makes the same decision on both and therefore incurs a false acceptance or false rejection. Hierarchy cannot recover information absent from the evidence channel.

4 Reference implementation

The v5.6.39 standard-library runtime implements atomic state updates, Plan/Phase creation, exactly one three-view DIC per Phase, DIC graph validation, Main-authored initial EPS plus disposable repair attempts, Action lineage, the four verification outcomes, package recovery, and prompt-tuning gates. Model-executed EPS nodes must record their actual model profile, execution surface, and applicable prompt-guidance profile before an Action can be recorded. The conformance matrix in Table 1 covers structural and governance behavior; it is not comparative agent-performance evidence.

A deterministic structural fixture changes one declared key in a seven-node workflow. The impact cone contains four nodes rather than all seven, demonstrating that the implementation computes the intended invalidation scope. The fixture assumes complete declarations and does not establish that language models will produce them reliably.

5 Evaluation programme

The central question is whether hierarchy’s relative benefit increases with effective task horizon. We vary dependency depth, branching, delayed dependencies, interacting constraints, and hidden intermediate faults across objectively graded repository, pipeline, and evidence-synthesis tasks.

The main conditions are C0 direct/reactive; C1 flat plan-and-execute; C2 contracted hierarchy; and C3 hierarchy plus verification-triggered repair/replanning. C1+V spends C3’s verifier calls on generic critique without local invalidation; C2-extra spends the same budget on additional generation. These controls distinguish the mechanism from extra tokens, calls, stronger prompts, and verifier-as-inference.

Primary outcomes are end-to-end success, hard-constraint satisfaction, unsupported completion, fault-detection latency, propagation size, recovery locality, and rework; costs include calls, tokens, tools, and wall time. The primary analysis tests a condition-by-effective-horizon interaction with matched task instances and mixed effects. A positive paper claim requires frozen prompts and graders, matched-budget analysis, at least two task families, and all primary conditions. Current unit/smoke evidence does not cross this gate.

6 Related work and positioning

Planning/search methods organize within-task reasoning [Yao et al., 2023b,a, Besta et al., 2024, Prasad et al., 2024, Zhou et al., 2024a]; compound-agent systems organize roles and conversations [Li et al., 2023, Wu et al., 2023, Chen et al., 2024, Hong et al., 2024]; memory systems externalize context [Packer et al., 2023, Wang et al., 2024]; and workflow optimizers search prompts or agent graphs [Khatab et al., 2024, Hu et al., 2025]. Our protocol is complementary: it makes authority, semantic refinement, evidence, invalidation, and human checkpoints persistent and inspectable across executions.

AgentBench, WebArena, GAIA, SWE-bench, OSWorld, and τ -bench document hard stateful tasks [Liu et al., 2024, Zhou et al., 2024b, Mialon et al., 2024, Jimenez et al., 2024, Xie et al., 2024, Yao et al., 2024]. We propose controlled graph/fault manipulation because benchmark score alone does not identify why a hierarchy helps. Scalable oversight and process supervision motivate verification but do not make verifiers sound [Lightman et al., 2023, Irving et al., 2018, Leike et al., 2018].

7 Limitations and responsible use

The mechanism adds overhead on short tasks. DICs can also shift rather than reduce human burden when they are verbose, poorly prioritized, or require frequent reconstruction; salience and compactness are therefore part of the interface contract. DICs and dependency footprints can be wrong, hidden state invalidates local recovery, and verifier errors may be correlated. Role separation does not guarantee independent judgment. The runtime is not a security boundary; prompt injection, malicious tools, and mutable external services require separate controls [Ruan et al., 2024, Debenedetti et al., 2024]. The formal cost model is stylized; human-attention intensity uses task-specific progress weights and imperfect proxies for cognitive effort; and no confirmatory performance or attention-efficiency result is claimed.

Responsible-use statement. The system is intended to make agent authority, evidence, and intervention points more visible, not to authorize unsupervised high-impact actions. Deployments should minimize credentials and external permissions, sandbox tools, preserve audit logs, require explicit human approval for destructive, release, submission, financial, privacy-sensitive, or other high-impact actions, and test for prompt injection and evidence manipulation. The workflow may create a false sense of assurance when contracts or verifiers are weak; users should treat residual uncertainty as part of the output.

8 Conclusion

A useful meta-agent needs more than another planning prompt. It needs a durable object that says what remains authoritative, which objective is primary, how intent was refined, what evidence supports progress, what a repair invalidates, and when human attention is actually required. Contracted hierarchies provide that object while separating a persistent semantic manager from repository-aware coding-agent orchestration. The implementation and theory establish a defensible mechanism and its limits; the frozen matched-budget study is the next gate for any claim that the system improves trustworthy long-horizon progress or lowers human attention required per unit of verified progress.

References

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI Conference on Artificial Intelligence*, 2024.

206 Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu,
207 Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. Agentverse: Facilitating multi-agent collaboration and
208 exploring emergent behaviors. In *International Conference on Learning Representations*, 2024.

209 John T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future*
210 *Generation Computer Systems*, 22(3):303–312, 2006.

211 Edoardo DeBenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian
212 Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for
213 llm agents. In *Advances in Neural Information Processing Systems*, 2024.

214 Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin
215 Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for
216 multi-agent collaborative framework. In *International Conference on Learning Representations*,
217 2024.

218 Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International*
219 *Conference on Learning Representations*, 2025.

220 Geoffrey Irving, Paul Christiano, and Dario Amodei. Ai safety via debate. *arXiv preprint*
221 *arXiv:1805.00899*, 2018.

222 Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik
223 Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International*
224 *Conference on Learning Representations*, 2024.

225 Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vard-
226 hamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, et al. Dspy: Compiling
227 declarative language model calls into self-improving pipelines. In *International Conference on*
228 *Learning Representations*, 2024.

229 Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. Scalable agent
230 alignment via reward modeling: A research direction. *arXiv preprint arXiv:1811.07871*, 2018.

231 Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem.
232 Camel: Communicative agents for mind exploration of large scale language model society. In
233 *Advances in Neural Information Processing Systems*, 2023.

234 Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan
235 Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint*
236 *arXiv:2305.20050*, 2023.

237 Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding,
238 Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International*
239 *Conference on Learning Representations*, 2024.

240 Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas
241 Scialom. Gaia: A benchmark for general ai assistants. In *International Conference on Learning*
242 *Representations*, 2024.

243 Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. In *Proceedings of*
244 *the ACM on Programming Languages*, 2018.

245 Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez.
246 Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

247 Archiki Prasad, Alexander Koller, Mareike Hartmann, Peter Clark, Ashish Sabharwal, and Tushar
248 Khot. Adapt: As-needed decomposition and planning with language models. In *Findings of the*
249 *Association for Computational Linguistics: NAACL*, 2024.

250 Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois,
251 Chris J. Maddison, and Tatsunori Hashimoto. Toolemu: Identifying the risks of lm agents with an
252 lm-emulated sandbox. In *International Conference on Learning Representations*, 2024.

- 253 Zhenhailong Wang, Shaoguang Liu, Yixin Chen, Zhaoran Zheng, Mingyuan Wang, Kenji Kawaguchi,
254 and Huaxiu Yao. Hiagent: Hierarchical working memory management for solving long-horizon
255 agent tasks with large language models. *arXiv preprint*, 2024.
- 256 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang,
257 Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent
258 conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- 259 Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Tenyue
260 Hua, Zhoujun Cheng, Dong-Suk Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal
261 agents for open-ended tasks in real computer environments. In *Advances in Neural Information*
262 *Processing Systems*, 2024.
- 263 Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik
264 Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In
265 *Advances in Neural Information Processing Systems*, 2023a.
- 266 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.
267 React: Synergizing reasoning and acting in language models. In *International Conference on*
268 *Learning Representations*, 2023b.
- 269 Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. tau-bench: A benchmark for
270 tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- 271 John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of*
272 *the ACM*, 17(9):530–531, 1974.
- 273 Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language
274 agent tree search unifies reasoning, acting, and planning in language models. In *International*
275 *Conference on Machine Learning*, 2024a.
- 276 Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng,
277 Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment
278 for building autonomous agents. In *International Conference on Learning Representations*, 2024b.